Multimodal Fine-Tuning with Small Language Models

Bounded By LLMs

PRIYANKA S — PES1UG23AM219

PADARTHI NEHA SAI — PES1UG23AM200

Project Overview

This project fine-tunes a **Vision-Language Model (ViT-GPT2)** on the **RICO-Screen2Words** dataset to automatically generate natural language captions for mobile app screenshots. The model learns to describe UI elements, layouts, and functionality of Android app screens in plain English.

Example:

Screenshot	Caption
App screenshot	*"a settings page with toggle buttons for notifications"*

Repository Structure

```
rico-screen-captioning/
│
├── notebook.ipynb        ← Full training + inference notebook
├── README.md             ← This file
└── requirements.txt      ← All dependencies
```

Dataset

RICO-Screen2Words 🔗
https://huggingface.co/datasets/rootsautomation/RICO-Screen2Words

Property	Details
Source	RICO UI dataset + crowdsourced captions
Total samples	~22,000
Samples used	500 (subset for T4 GPU)
Train split	400 samples (80%)
Val split	100 samples (20%)
Image type	Android app screenshots
Caption type	Single natural language sentence

Each example contains:

- `image` — screenshot of a mobile UI screen
- `captions` — list of 5 human-written descriptions

Why a subset? Training on the full 22,000 samples would exceed T4 GPU memory and time limits on Google Colab. 500 samples is sufficient for a proof-of-concept fine-tune.

Model

Base Model: `nlpconnect/vit-gpt2-image-captioning` 🔗
https://huggingface.co/nlpconnect/vit-gpt2-image-captioning

Fine-tuned Model: `pnehasai/rico-screen-caption-model` 🔗
https://huggingface.co/pnehasai/rico-screen-caption-model

Architecture

```
Screenshot ──▶ ViT Encoder ──▶ GPT-2 Decoder ──▶ Caption
               (Image →        (Features →
                Features)       Text)
```

ViT (Vision Transformer) — Encoder

- Splits each screenshot into **16×16 pixel patches**
- Each patch is converted into a high-dimensional feature vector
- Learns spatial relationships between UI elements across the image
- Produces a sequence of image embeddings passed to the decoder

GPT-2 — Decoder

- Receives image embeddings from ViT as context
- Generates caption **word by word** in an auto-regressive manner
- Each word is predicted based on the image features and all previously generated words
- Stops at the end token or when `max_length=32` is reached

Why ViT-GPT2?

Property	Details
Model size	~330M parameters — lightweight SLM
Input	Images (224×224 pixels)
Output	Natural language captions
GPU requirement	T4 compatible
Pretrained on	MS-COCO, Conceptual Captions
Architecture type	Vision Encoder-Decoder

Why this model and not others?

- It is a proven, stable vision-language model that runs reliably on T4 GPU
- The encoder-decoder architecture is well suited for image-to-text generation tasks
- It is lightweight enough for fine-tuning on a subset of data within Colab time limits
- The pretrained weights already encode general visual understanding, so fine-tuning on RICO screenshots allows it to adapt to the specific domain of mobile UI captioning quickly

---

Training Strategy

Parameter	Value	Reason
Samples	500	Fits T4 GPU memory within Colab time limits

Train/Val split	80/20	Standard split for small datasets
Epochs	1	Avoids overfitting on small dataset
Learning rate	5e-5	Standard starting point for vision-language fine-tuning
Batch size	4	Fits T4 16GB VRAM without memory errors
Max caption length	32 tokens	RICO captions are short UI descriptions, 32 is sufficient
Optimizer	AdamW	Best suited for transformer fine-tuning due to weight decay
FP16	Enabled	Halves memory usage and speeds up training on GPU
Fine-tuning type	Direct (full model)	All weights updated end-to-end
Padding token	Masked with -100	Ensures loss is not computed on padding, only real tokens

Why Direct Fine-Tuning?

Direct fine-tuning updates all model weights rather than only a small adapter layer. This gives the model a stronger signal to adapt from natural image captions (what it was pretrained on) to mobile UI screenshot descriptions (the new domain). Given the small dataset size, 1 epoch is enough to shift the model toward the new domain without overfitting.

Why AdamW?

AdamW extends the Adam optimizer with proper weight decay, which prevents individual weights from growing too large during fine-tuning. This is especially important when adapting a large pretrained model on a small dataset, as it acts as a regularizer and keeps the model generalizable.

Why FP16?

Mixed precision training (FP16) stores activations in 16-bit float instead of 32-bit, cutting memory usage roughly in half. This allows a larger effective batch size on T4 GPU and speeds up matrix multiplications, with no meaningful loss in accuracy.

Pipeline

1. Install Dependencies
   ↓
2. Load RICO-Screen2Words Dataset
   ↓
3. Visualize Sample (image + caption)
   ↓
4. Load Pretrained ViT-GPT2 Model
   ↓
5. Preprocess
   - Convert images → pixel values (224×224, normalized)
   - Tokenize captions → token IDs
   - Mask padding tokens with -100
   ↓
6. Collate Function
   - Stack pixel_values into batch tensors
   - Stack labels into batch tensors
   ↓
7. Fine-tune Model (Manual Training Loop)
   - Forward pass → compute cross-entropy loss
   - Backward pass → gradients computed
   - AdamW updates all model weights
   - Log loss every 20 steps
   ↓
8. Save Model Locally
   ↓
9. Upload to HuggingFace Hub
   ↓
10. Run Inference from HuggingFace
   ↓
11. Evaluate with BLEU Score

Preprocessing in Detail

Each image is resized to **224×224 pixels** and normalized using the ViT feature extractor's built-in mean and standard deviation values. This ensures pixel values are in the range expected by the pretrained ViT encoder. Each caption is tokenized using GPT-2's byte-pair encoding (BPE) tokenizer, padded or truncated to a fixed length of 32 tokens. Padding token positions are replaced with -100 so the cross-entropy loss function ignores them during training.

Training Loop in Detail

At each step the model receives a batch of 4 image-caption pairs. The ViT encoder processes the images into feature embeddings, which the GPT-2 decoder uses to predict the caption tokens one by one. The loss is computed as the average cross-entropy between predicted token probabilities and the actual token labels. Gradients are backpropagated through both the decoder and encoder, and AdamW updates all weights accordingly.

Results

BLEU Scores

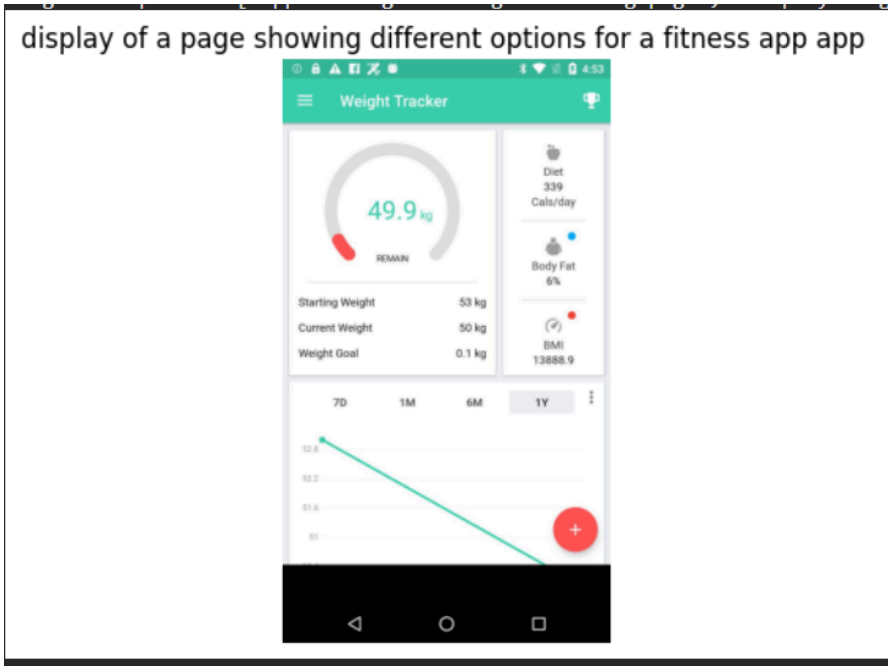
Metric	Score	What it Measures
BLEU-1	0.1758	Individual word overlap
BLEU-2	0.0928	2-word phrase overlap
BLEU-4	0.0312	4-word phrase overlap (strictest)

Evaluated on 50 validation samples. Scores reflect lightweight fine-tuning on 400 samples for 1 epoch on T4 GPU.

Why are scores relatively low?

- Only 500 training samples covering a fraction of the 200+ app categories in RICO
- Only 1 epoch of training — the model has limited exposure to UI-specific language
- The pretrained model was trained on natural photos, not mobile screenshots — domain gap exists
- Scores would improve significantly with more data (5000+ samples) and more epochs (3–5)

Sample Prediction:



#	Predicted Caption	Actual Caption
1	`display of a page showing different options for a fitness app app`	`['app showing the weight tracking page', 'display image', 'page displaying the profile information in fitness app', 'page showing weight tracker on app', 'weight tracker page displayed of an fitness app']`

Quick Inference

Pull the fine-tuned model from HuggingFace and caption any mobile screenshot in seconds. Full inference code is available in `notebook.ipynb`.

Model on HuggingFace: https://huggingface.co/pnehasai/rico-screen-caption-model

---

Requirements

transformers
datasets
pillow

accelerate
huggingface_hub
scikit-learn
torch
nltk

Install all:

pip install transformers datasets pillow accelerate huggingface_hub scikit-learn nltk

---

Links

Resource	Link
Fine-tuned Model	https://huggingface.co/pnehasai/rico-screen-caption-model
Dataset	https://huggingface.co/datasets/rootsautomation/RICO-Screen2Words
Base Model	https://huggingface.co/nlpconnect/vit-gpt2-image-captioning
Colab Notebook	*BoundedByLLMs*